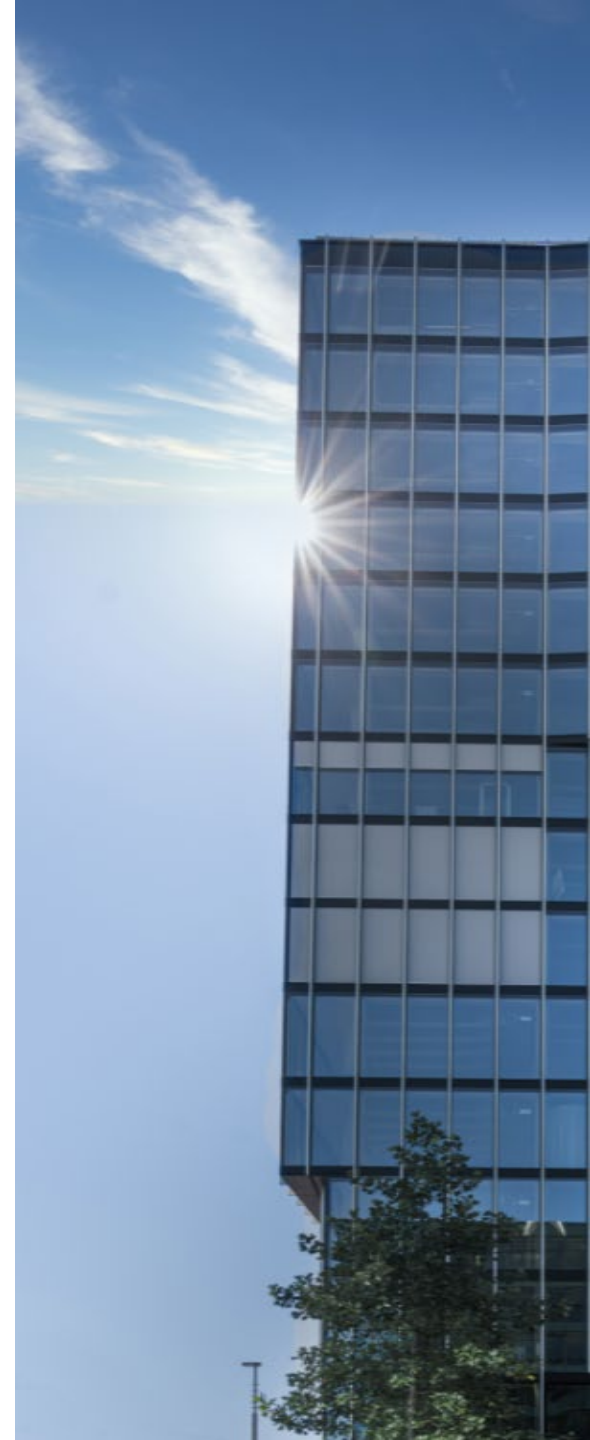


Objektorientierte Programmierung

Grundlagen der GUI-Programmierung

Graphical **U**ser **I**nterface

Roland Gisler



Inhalt

- Grundlagen – Graphical User Interfaces und Objektorientierung
- Struktureller Aufbau eines GUI
- Layout-Konzept
- Ereignisgesteuerte Programmierung
- Model View Controller (MVC)
- Zusammenfassung

Lernziele

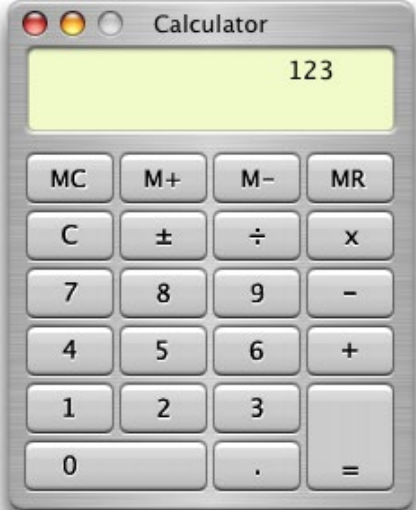
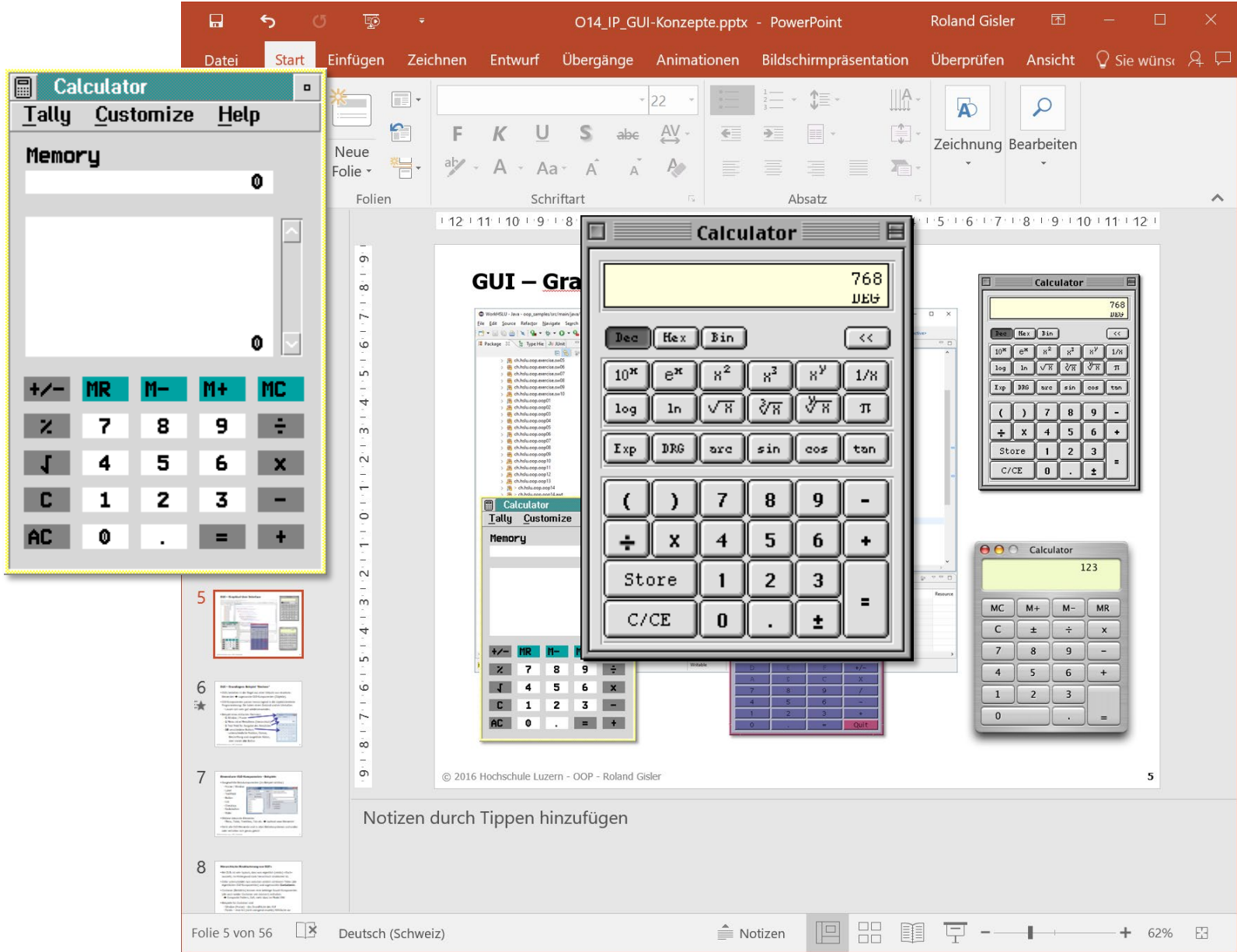
- Sie kennen grundlegende Konzepte wie GUIs programmiert werden.
- Sie können die verschiedenen Herausforderungen der GUI-Programmierung anhand von Beispielen erläutern.
- Sie verstehen die Absicht des Model-View-Control (MVC) Prinzips.

GUI – Graphical User Interface

<https://de.wikipedia.org/wiki/Cuy>

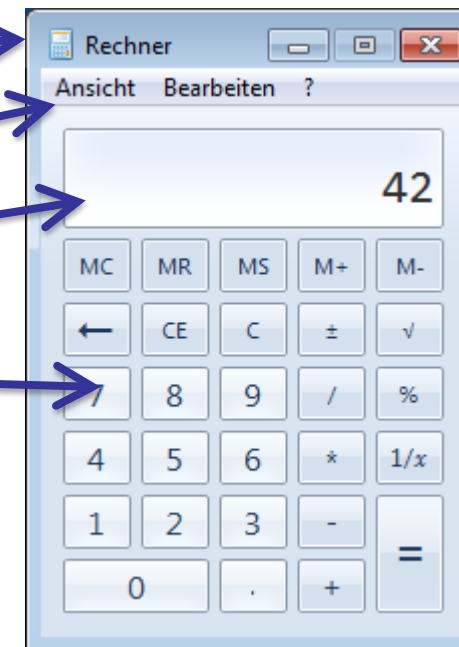


GUI – Graphical User Interface



GUI – Grundlagen: Beispiel "Rechner"

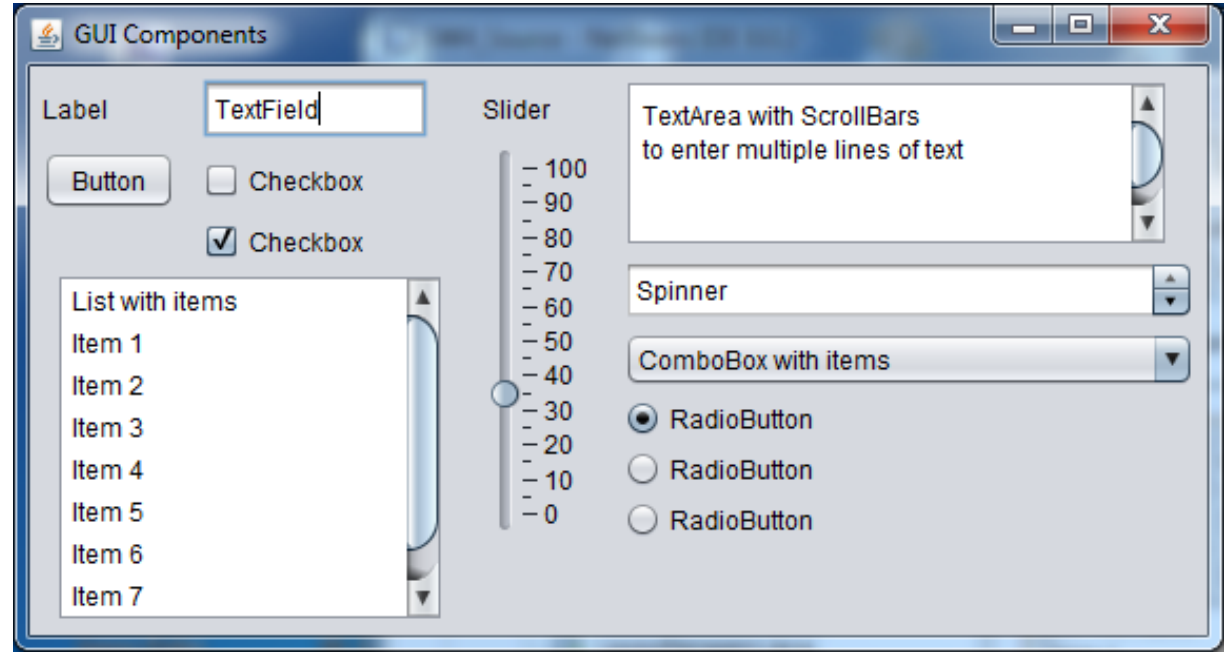
- GUIs bestehen in der Regel aus einer Vielzahl von einzelnen Elementen, sogenannte GUI-Komponenten (Objekte).
- GUI-Komponenten passen hervorragend in die objektorientierte Programmierung: Sie haben einen Zustand und ein Verhalten.
 - Lassen sich sehr gut wiederverwenden.
- Beispiel eines einfachen Rechners:
 - **1** Window / Frame
 - **1** Menu mit **n** MenuItems (hierarchisch)
 - **1** Text Field für Ausgabe des Resultates
 - **28** verschiedene Buttons
 - unterschiedliche Position, Grösse, Beschriftung und ausgelöste Aktion, aber immer **ein** Button



Elementare GUI-Komponenten - Beispiele

- Ausgewählte Basiskomponenten (im Beispiel sichtbar)

- Frame / Window
- Label
- TextField
- Button
- List
- Checkbox
- Radiobutton
- Slider



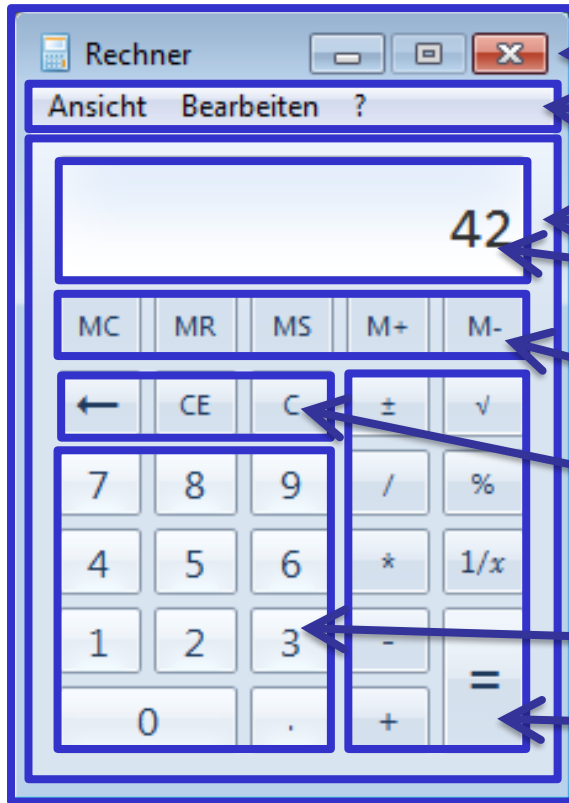
- Es gibt viele weitere Elemente:
 - Menu, Table, TreeView, Tab etc. → es gab/gibt laufend neue Elemente!
- Nicht alle GUI-Elemente sind in allen Betriebssystemen vorhanden und verhalten sich auch nicht überall genau gleich!

Hierarchische Strukturierung von GUI's

- Bei GUIs ist es sehr typisch, dass das, was eigentlich (relativ) «flach» aussieht, im Hintergrund stark hierarchisch strukturiert ist.
- Dafür unterscheidet man zwischen sichtbaren Teilen (die eigentlichen GUI-Komponenten) und sogenannten **Containern**.
- Container (Behälter) können eine beliebige Anzahl Komponenten (die selber wiederum Container sein können!) enthalten.
 - ➔ Composite Pattern, GoF, mehr dazu im Modul VSK
- Beispiele für Container sind
 - Window (Frame) – die Grundfläche einer GUI-Applikation.
 - Panels – eine (nicht zwingend visuelle) Hilfsfläche zur Zusammenfassung von Einzelkomponenten.

Beispiel: Rechner-GUI – konzeptionelle Idee

- Das GUI eines einfachen Taschenrechners könnte z.B. wie folgt strukturiert sein:

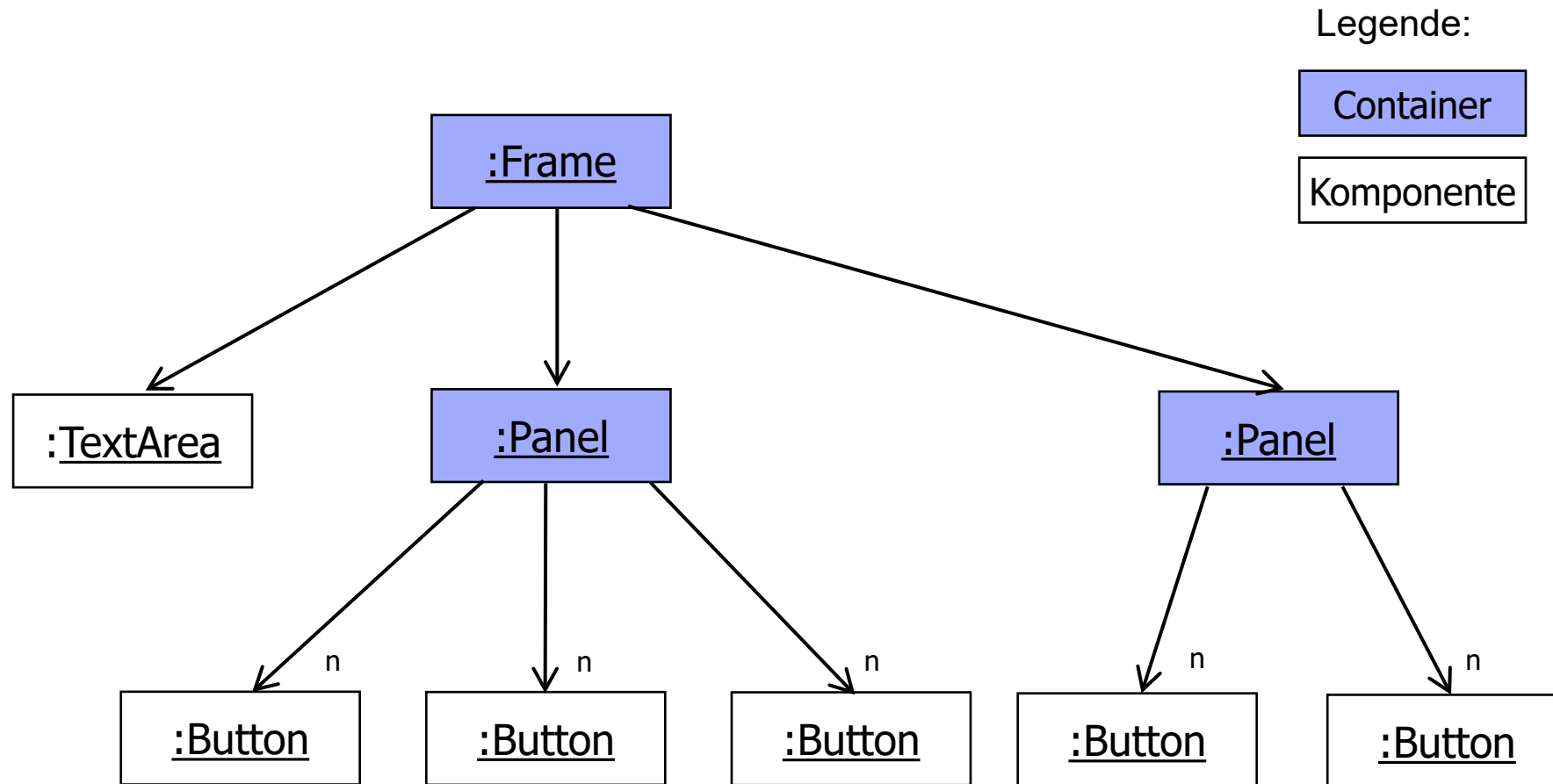


- Fenster (Frame)
- Menu-Bar
- Content-Panel (Root Panel)
- Display-Panel (mit TextField)
- Eingabebereiche (je: Panel mit Buttons)
 - Memory-Control
 - Input-Control
 - Dezimal-Pad
 - Operation-Control

➔ Hierarchische Struktur

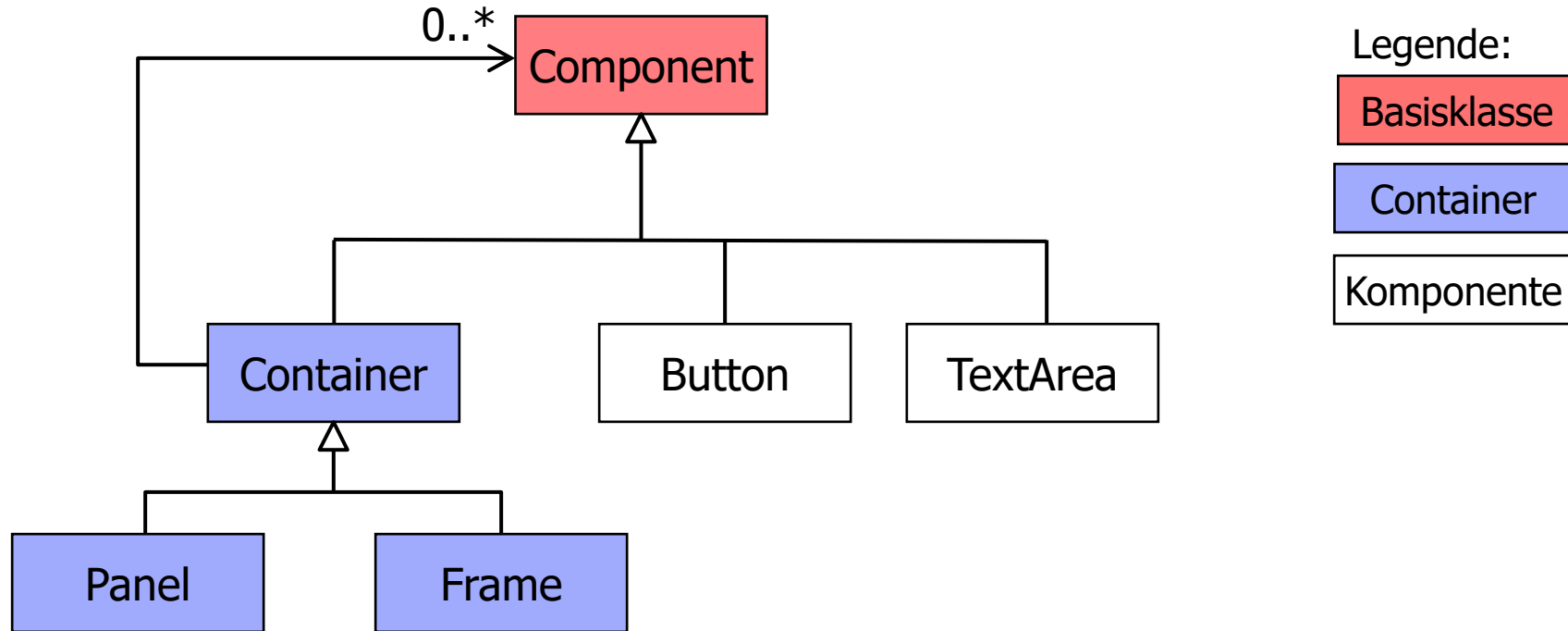
Objektdiagramm (konzeptionell)

- Reduziertes Diagramm zur Illustration der hierarchischen Objekt-Beziehungen:



Klassendiagramm (konzeptionell)

- Composite-Pattern (Kompositum) nach GoF:
Ein Container kann Komponenten oder aber wiederum Container enthalten.



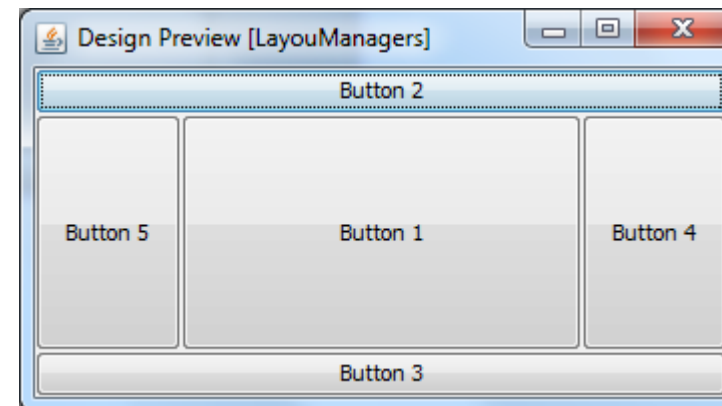
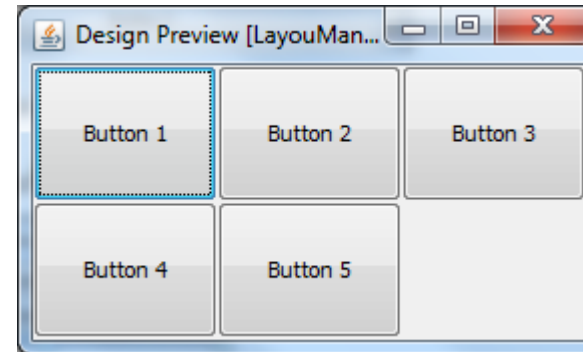
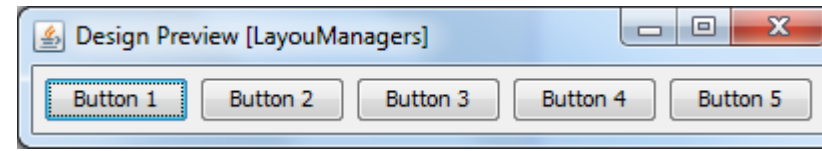
Layout-Konzept

Positionierung von Komponenten in einem Dialog

- Fenstersysteme haben sehr unterschiedliche Desktop-Auflösungen, Schriftgrößen und -arten, Abstände, Ränder usw.
- **Pixelgenaue** Positionierung und Dimensionierung der Komponenten ist längst **keine adäquate Lösung** mehr!
 - In den 90er-Jahren war das z.B. auf Windows unter Standard-VGA, SVGA oder XGA-Auflösungen durchaus üblich (96dpi).
 - Ebenso erinnern sich manche vielleicht noch an die 1-Pixel-GIFs im Webbereich (Pre-CSS-Ära).
- Plattformübergreifend (verschiedene OS) und auch mit HiDPI-Bildschirmen wird es zunehmend schwierig: Skalierungen von bis zu 200% und mehr werden nötig!
- Lösung: Konzept der **Layouts** mit **Constraints**.

Konzept der Layout-Manager

- Man «beschreibt» die Anordnung der einzelnen Elemente durch die Organisation mittels sogenannter Layouts. Drei **Beispiele**:
- **Flow**-Layout: Ein Element nach/neben dem anderen
 - horizontal oder vertikal.
- **Grid**-Layout: Ein- und Anordnung der Elemente in eine virtuelle Gitterstruktur.
- **Border**-Layout: Aufteilung in verschiedene Regionen, die bei einem Resize nicht alle gleichmässig mitwachsen.



Layout-Manager

- Die verschiedenen Layout-Manager werden in der Regel mit- und ineinander (hierarchisch) verknüpft.
- Müssen mit Bedingungen (Constraints) konfiguriert werden, welche Abstände und Ränder zwischen und um Komponenten festlegen.
- Jedes Element hat eine minimale, optimale und maximale Grösse, abhängig von verwendeten Schriften und Auflösungen.
- Die Elemente organisieren sich innerhalb des Layouts selber und adaptieren ihre Grösse bei einem Resize (Grössenveränderung z.B. eines Dialoges/Fensters).
- Herausforderung: Benötigt einiges an Planung und Erfahrung!
 - Kein schnelles «Pinself» von (ansprechenden) GUIs möglich!

Ereignisgesteuerte Programmierung

Bisher: Rein sequenzielle Programmierung

- Sequenzielle Aneinanderreihung von einzelnen Anweisungen
 - Start (bei Java) ausgehend von der `main()`-Methode.
- Laufen in einer vorgesehenen Abfolge (mit allfälligen Wiederholungen und Verzweigungen) reproduzierbar der Reihe nach ab.
 - Benutzer*in kann (wenn überhaupt) nur an explizit dafür vorgesehenen Stellen (und Momenten) in den Ablauf eingreifen.
- Entspricht häufig dem klassischen **EVA**-Prinzip
 - **E**ingabe, **V**erarbeitung und **A**usgabe.
- Beispiel: Drehorgel
 - Walze (oder Lochkarte) gibt den exakten sequenziellen Ablauf (hier: Töne) vor.



Ereignisgesteuerte Programmierung

- Sehr typische Technik für GUI-Programmierung (aber nicht nur!).
- Die meiste Zeit **wartet** das Programm.
- Erst wenn ein Ereignis (Event) eintritt, geschieht etwas.
 - Tastendruck, Mausklick, Touchgeste, Fusstritt etc.
- Das Ereignis löst dann eine (vor-)bestimmte Aktion aus.
 - Die Reihenfolge der Ereignisse kann völlig frei wählbar sein, oder aber auch schrittweise zu einem Ziel führen.
- Die effektive Bearbeitung kann auch im/in Hintergrund (-prozessen) und Nebenläufig stattfinden.



Ereignisgesteuerte Programmierung: Event/Listener!

- Die GUI-Programmierung funktioniert in Java massgeblich mit dem bereits bekannten **→Observer**-Pattern per Event/Listener-Modell.
 - Dadurch erreicht man eine möglichst **lose Kopplung** zwischen dem Auslöser einer Aktion und der Aktion selber.
- Ein populärer (GUI-)Event in Java ist z.B. der **ActionListener**, der für unmittelbare Aktionen wie z.B. das Drücken eines Buttons genutzt wird:
 - Event-Quelle **→** z.B. **Button**-Objekt
 - Event «click» **→** **ActionEvent**-Objekt wird gefeuert.
 - Event-Verarbeitung **→** Beliebiges Objekt, welches das **ActionListener**-Interface implementiert hat, und in der Methode **actionPerformed(...)** auf das Ereignis reagieren kann.



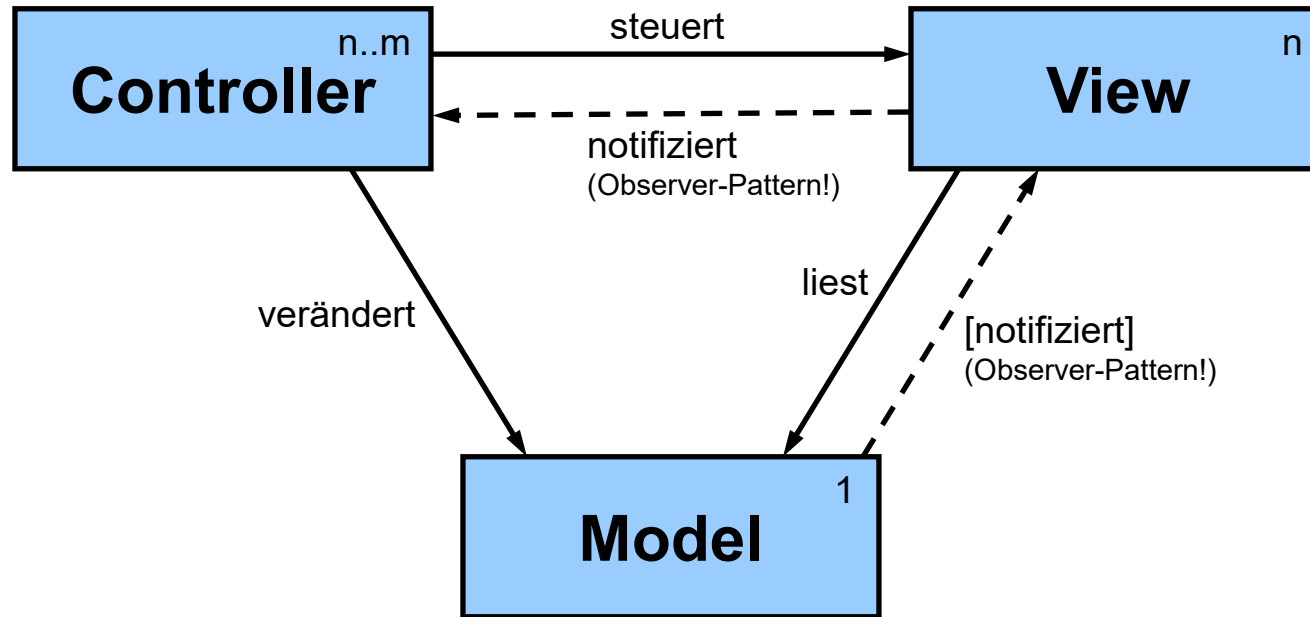
Model – View – Control (MVC)

Model – View – Control (MVC)

- Nach den Prinzipien SRP und SoC sollte man Aufgaben möglichst strikte voneinander trennen, um lose Kopplung, hohe Kohäsion, eine bessere Wiederverwendung (und viele weitere Vorteile) zu erreichen.
- Bei der GUI-Programmierung kommen viele Dinge zusammen:
Man steuert (kontrolliert), welche Informationen (Datenmodelle) auf welche Art wo und wann dargestellt werden.
- Gemäss SRP versucht man, mindestens diese drei Teile möglichst gut voneinander zu trennen:
 - **Model** – Die eigentlichen Datenobjekte, z.B. eine **Person**.
 - **View** – Eine [G]UI-Komponente, welche die Daten anzeigt.
 - **Control** – Eine (meist übergeordnete) Steuerung, welche das Ganze (bzw. mindestens **M** und **V**) koordiniert.

MVC - Grundidee

- Vereinfachtes, konzeptionelles Blockdiagramm (nicht UML):



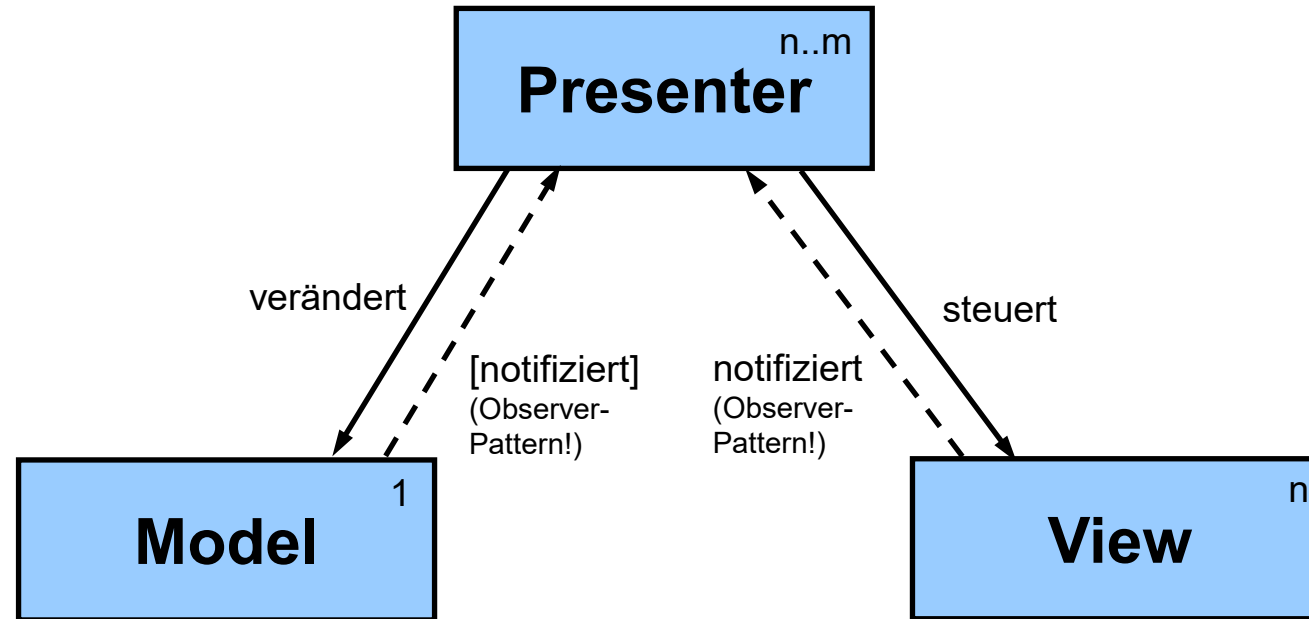
- Die Pfeile zeigen immer in Richtung der Abhängigkeit:
 - **Model**: Ist selber völlig **unabhängig** und wird «nur» verwendet.
 - **View**: Es sind verschiedene **Varianten** für ein und dasselbe Model möglich.
 - **Control**: Verbindet das Modell und die View(s), **steuert** und **koordiniert** diese.

MVC - Hintergrund

- MVC ist ein sehr fundamentales Konzept, das typisch auf mehreren, unterschiedlichen Abstraktionsebenen wiederholt eingesetzt werden kann.
 - Häufig hierarchisch verschachtelt!
- Es gibt nicht eine einzige, richtige Form und Implementation!
 - MVC wird je nach Situation, Technologie und Absicht passend adaptiert.
 - Dynamik der (Anzahl von) Views, der Änderungen von Daten etc.
- Es haben sich verschiedenste Varianten und Verfeinerungen weiterentwickelt:
 - Zum Beispiel: MVC2, MVVM, MVP – kann teilweise verwirrend sein.
- Gemeinsamkeit: Ziel ist immer die «Gewaltentrennung» zwischen dem Modell (Daten), dessen Visualisierung (View) und deren Steuerung und Koordination (Control) →SRP.
 - Das Modell ist beständig und unabhängig, die View abhängig vom Modell und der Controller ist meistens sehr stark spezialisiert.

Variante: Model – View – Presenter (MVP)

- Vereinfachte, konzeptionelle Darstellung (nicht UML):



- Die Pfeile zeigen in die Richtung der Abhängigkeit:
 - **Model**: Ist völlig unabhängig und wird «**nur**» verwendet.
 - **View**: Es sind verschiedene **Varianten** für ein und dasselbe Model möglich.
 - **Presenter**: **Vermittelt** zwischen Modell und View und entkoppelt diese vollständig.

Zusammenfassung

- Objektorientierte Strukturierung, Wiederverwendung von Komponenten und Containern, Composite-Pattern, hierarchisch.
- (Möglichst) plattformunabhängige Implementation, Verwendung von Layout-Managern.
- Ereignisorientierte Programmierung mit z.B. mit Observer-Pattern.
- Designkonzepte wie MVC (Model-View-Control) oder MVP (Model-View-Presenter) zur «Gewaltentrennung» der unterschiedlichen Aufgaben.



Fragen?